Modul 2: Ekosistem Hadoop

1 Tujuan

Modul ini bertujuan untuk memberikan pemahaman konseptual dan implementatif mengenai arsitektur serta komponen utama dalam ekosistem Hadoop sebagai fondasi sistem Big Data terdistribusi. Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

1. Memahami arsitektur dan komponen utama ekosistem Hadoop, termasuk peran *NameNode*, *DataNode*, *ResourceManager*, dan *NodeManager*, serta hubungan antara HDFS, YARN, dan MapReduce dalam sistem terdistribusi.

2. Mampu melakukan pengelolaan data pada *Hadoop Distributed File System (HDFS)* melalui operasi dasar seperti unggah, unduh, inspeksi direktori, serta memahami mekanisme pembagian blok, replikasi data, dan toleransi kesalahan.

3. Menganalisis mekanisme pemrosesan data menggunakan paradigma *MapReduce* dan manajemen sumber daya melalui YARN, serta mengevaluasi keterbatasannya dibandingkan dengan pendekatan komputasi Big Data modern.

2 Konsep Dasar

2.1 Apache Hadoop

Apache Hadoop merupakan kerangka kerja *open-source* berbasis Java yang dirancang untuk mendukung pemrosesan kumpulan data berskala besar dengan pola akses *streaming* pada lingkungan komputasi terdistribusi berbasis klaster. Hadoop memungkinkan penyimpanan data dalam volume yang sangat besar, baik data terstruktur, semi-terstruktur, maupun tidak terstruktur, melalui sistem berkas terdistribusi (*Distributed File*

System/DFS), serta memprosesnya secara paralel pada banyak node.

Sebagai platform penyimpanan dan pemrosesan data, Hadoop memiliki tingkat skalabilitas yang tinggi serta efisiensi biaya yang signifikan. Skalabilitas Hadoop merujuk pada kemampuannya untuk mempertahankan kinerja sistem meskipun terjadi peningkatan beban kerja secara drastis, yang dicapai melalui penambahan node baru ke dalam klaster. Dalam arsitekturnya, berkas pada Hadoop menerapkan prinsip *write-once, read-many*, yaitu data hanya dapat ditulis satu kali dan dibaca berulang kali tanpa memungkinkan perubahan terhadap isi berkas setelah disimpan.

Klaster didefinisikan sebagai sekumpulan komputer yang saling terhubung dan bekerja secara terkoordinasi sehingga berfungsi sebagai satu sistem terpadu. Klaster Hadoop dirancang untuk menyimpan dan menganalisis data dalam jumlah besar yang bersifat heterogen dalam lingkungan komputasi terdistribusi secara efisien dan ekonomis.

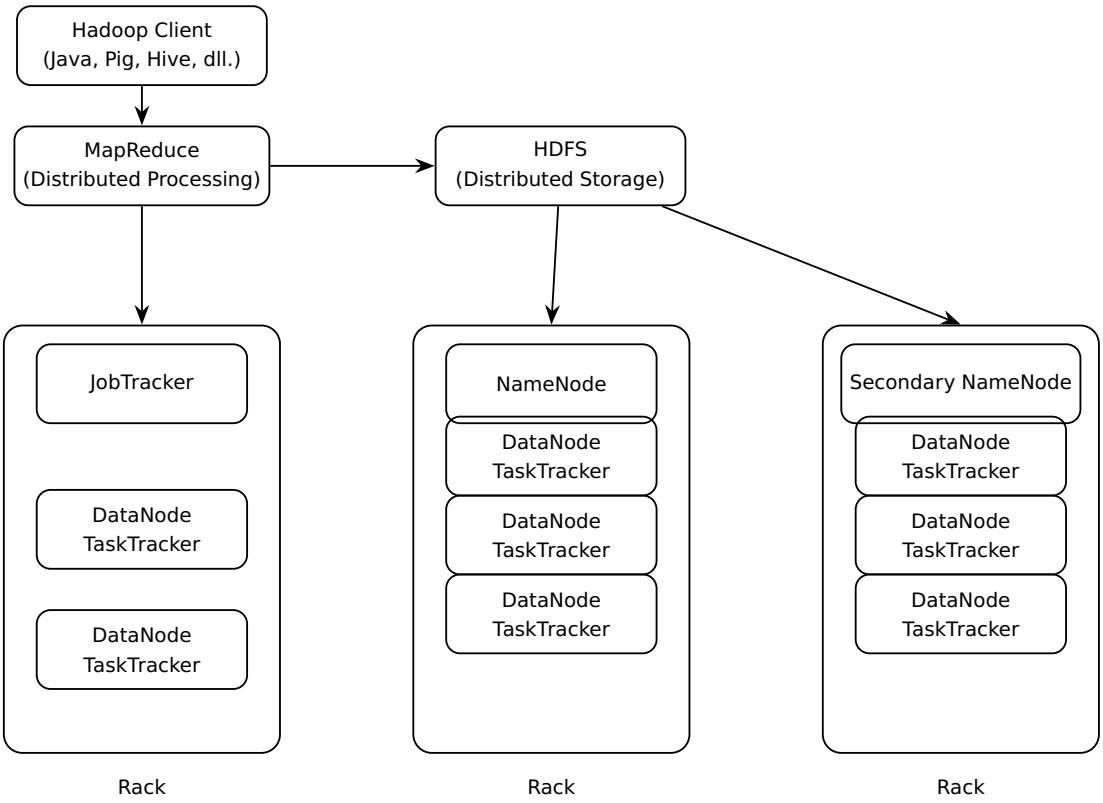
Arsitektur Apache Hadoop secara konseptual terdiri atas dua lapisan utama, yaitu lapisan penyimpanan dan lapisan pemrosesan. Lapisan penyimpanan direpresentasikan oleh Hadoop Distributed File System (HDFS), sedangkan lapisan pemrosesan berada di atasnya dalam bentuk mesin MapReduce. Struktur berlapis ini memungkinkan pemisahan tanggung jawab antara pengelolaan data dan eksekusi komputasi terdistribusi, sehingga meningkatkan efisiensi dan skalabilitas sistem secara keseluruhan.

HDFS berperan sebagai sistem penyimpanan terdistribusi yang dirancang untuk menangani data dalam skala besar dengan tingkat toleransi kesalahan yang tinggi. Komponen utama pada HDFS meliputi *NameNode*, yang bertanggung jawab atas pengelolaan metadata sistem berkas, serta *DataNode*, yang menyimpan blok-blok data secara fisik di setiap node dalam klaster. Selain itu, terdapat *Secondary NameNode* yang berfungsi membantu proses pengelolaan metadata melalui mekanisme penggabungan (*checkpointing*), meskipun tidak berperan sebagai pengganti langsung *NameNode*.

Pada lapisan pemrosesan, MapReduce bertindak sebagai kerangka kerja komputasi terdistribusi yang memproses data secara paralel pada berbagai node dalam klaster. Komponen *JobTracker* bertugas mengoordinasikan dan menjadwalkan pekerjaan (*job*) yang dikirimkan oleh klien Hadoop, sedangkan *TaskTracker* pada masing-masing *DataNode* menjalankan tugas-tugas pemrosesan yang telah dibagi. Dengan mekanisme ini,

Hadoop mampu mendistribusikan beban kerja secara merata ke berbagai node dalam klaster.

Secara fisik, node-node dalam Hadoop dikelompokkan ke dalam unit yang disebut *rack*. Setiap rack terdiri atas sejumlah *DataNode* yang saling terhubung dalam satu segmen jaringan. Pengelompokan berbasis rack ini mendukung strategi replikasi data dan optimasi komunikasi jaringan, sehingga meningkatkan keandalan serta kinerja sistem dalam lingkungan komputasi terdistribusi.



Gambar 1: *Arsitektur Apache Hadoop*

Hadoop Ekosistem

Ekosistem Hadoop terdiri atas empat lapisan utama, yaitu:

1. *Data Storage Layer*
2. *Data Processing Layer*
3. *Data Access Layer*
4. *Data Management Layer*

Keempat lapisan ini membentuk arsitektur terintegrasi yang mendukung penyimpanan, pemrosesan, akses, serta pengelolaan data dalam lingkungan komputasi terdistribusi.

Data Storage Layer mencakup HDFS dan HBase. HDFS berfungsi sebagai sistem berkas terdistribusi yang menyimpan data dalam lingkungan klaster. Sementara itu, HBase merupakan basis data berorientasi kolom yang digunakan untuk menyimpan data terstruktur dalam skala besar.

Data Processing Layer terdiri atas MapReduce dan YARN. MapReduce bertanggung jawab terhadap eksekusi dan pemrosesan pekerjaan (*job*) secara paralel, sedangkan YARN menangani alokasi sumber daya, penjadwalan pekerjaan, serta mekanisme pemantauan sistem.

Data Access Layer meliputi Hive, Pig, Mahout, Avro, dan SQOOP. Hive merupakan bahasa kueri yang digunakan untuk mengakses data dalam HDFS. Pig adalah bahasa skrip tingkat tinggi untuk analisis data. Mahout menyediakan platform untuk komputasi *machine learning*. Avro berfungsi sebagai kerangka kerja serialisasi data. SQOOP merupakan alat untuk mentransfer data antara basis data relasional tradisional dan HDFS, baik ke maupun dari HDFS.

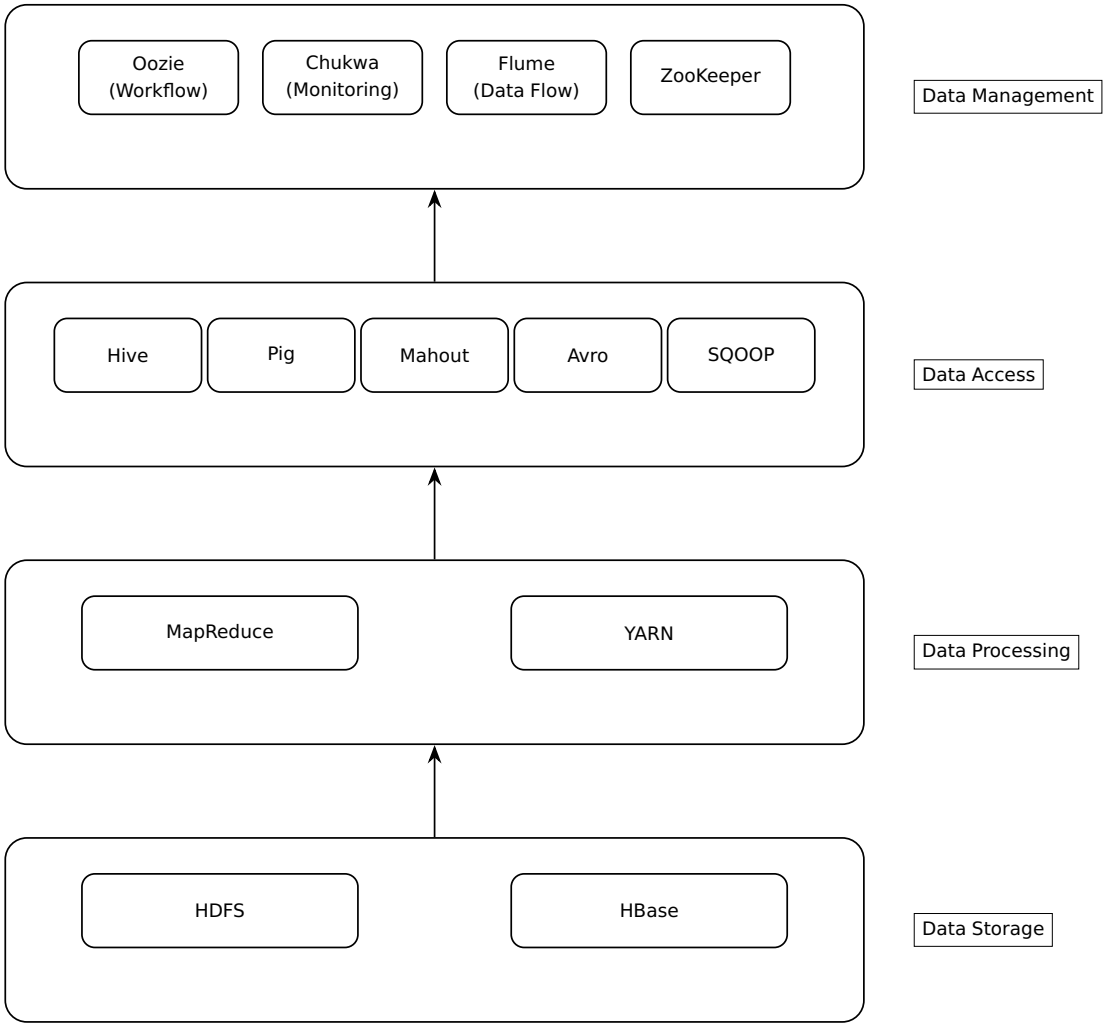
Data Management Layer berinteraksi langsung dengan pengguna akhir dan mencakup Oozie, Chukwa, Flume, serta ZooKeeper. Oozie berfungsi sebagai *workflow scheduler*. Chukwa digunakan untuk pengumpulan dan pemantauan data. Flume bertugas mengalirkan data dari berbagai sumber menuju HDFS. ZooKeeper menyediakan layanan koordinasi terdistribusi untuk menjaga sinkronisasi antar komponen dalam ekosistem Hadoop.

2.2 Hadoop Storage

HDFS (Hadoop Distributed File System)

Hadoop Distributed File System (HDFS) dirancang untuk menyimpan kumpulan data berskala besar dengan pola akses *streaming* pada perangkat keras komoditas (*low-cost commodity hardware*). Arsitektur HDFS tidak bergantung pada perangkat keras berbiaya tinggi dengan tingkat keandalan yang sangat tinggi, melainkan mengandalkan mekanisme replikasi dan distribusi data untuk menjamin ketahanan terhadap kegagalan (*fault tolerance*).

Data yang dihasilkan dari berbagai sumber disimpan dalam HDFS dengan prinsip *write-once, read-many*, di mana data ditu-



Gambar 2: *Hadoop Ecosystem*

lis satu kali dan dapat dibaca berulang kali untuk keperluan analisis. Proses analisis tersebut bertujuan untuk mengekstraksi informasi dan pengetahuan dari kumpulan data yang tersimpan. Namun demikian, HDFS tidak dirancang untuk aplikasi yang membutuhkan latensi rendah (*low latency access*). Untuk kebutuhan tersebut, HBase lebih sesuai digunakan karena mendukung akses data dengan waktu respons yang lebih cepat.

Secara teknis, HDFS menyimpan data dengan membaginya menjadi blok-blok berukuran lebih kecil. Setiap blok dari suatu berkas direplikasi pada beberapa node yang berbeda guna menjamin ketersediaan dan ketahanan sistem terhadap kegagalan. Apabila terjadi kerusakan blok, kegagalan disk, atau gangguan pada suatu mesin, blok data dapat dipulihkan melalui salinan replikasi yang tersimpan pada mesin lain yang terpisah secara fisik dalam klaster.

Mengapa HDFS?

Keunggulan utama HDFS terletak pada kemampuannya meningkatkan kinerja melalui mekanisme distribusi dan paralelisme. Sebagai ilustrasi, apabila 500 GB data dibaca pada satu mesin tunggal yang memiliki empat kanal I/O dengan kecepatan masing-masing 100 MB/s, maka waktu yang dibutuhkan untuk membaca seluruh data tersebut adalah sekitar:

$$\frac{500 \times 1024 \text{ MB}}{4 \times 100 \text{ MB/s}} = 1280 \text{ detik} \approx 21{,}3 \text{ menit.}$$

Waktu tersebut belum termasuk proses analisis data yang akan semakin menambah total durasi komputasi.

Sebaliknya, apabila data yang sama didistribusikan ke dalam 100 mesin dengan spesifikasi I/O serupa, maka proses pembacaan dapat dilakukan secara paralel. Secara teoritis, waktu yang dibutuhkan menjadi:

$$\frac{1280 \text{ detik}}{100} \approx 12{,}8 \text{ detik.}$$

Pendekatan terdistribusi ini menghilangkan *bottleneck* yang umumnya terjadi pada sistem terpusat dan secara signifikan meningkatkan performa. Inilah prinsip dasar yang diimplementasikan Hadoop melalui Distributed File System (DFS), di mana data disimpan pada ratusan *DataNode* dan proses pengambilan data berlangsung secara paralel di seluruh klaster.

HDFS Architecture

HDFS dirancang dengan tingkat *fault tolerance* yang tinggi dan diimplementasikan pada perangkat keras komoditas. Sistem ini mendukung penyimpanan data dalam skala sangat besar, mulai dari terabyte hingga petabyte, serta dirancang agar mudah dipindahkan (*portable*) antar platform. Secara arsitektural, HDFS mengadopsi model *master/slave*, di mana satu mesin dalam klaster bertindak sebagai *master* dan mesin lainnya berperan sebagai *slave*.

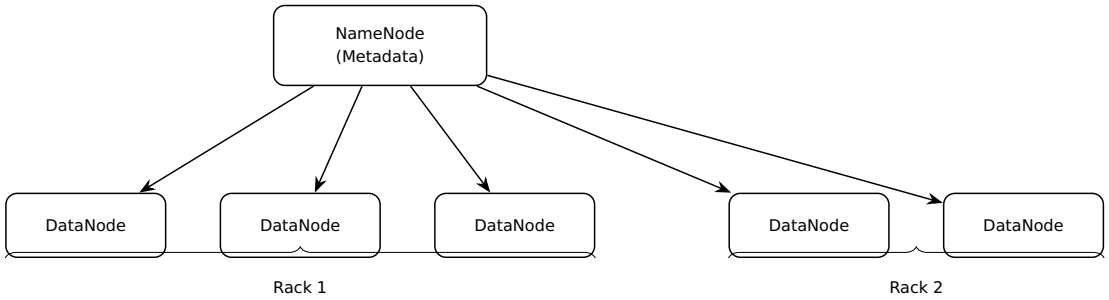
Node master menjalankan *NameNode*, yang bertanggung jawab atas pengelolaan *namespace* sistem berkas secara keseluruhan, pemantauan kesehatan *DataNode* melalui sinyal *heartbeat*, serta pengendalian akses berkas oleh pengguna. NameNode tidak menyimpan data aktual, melainkan metadata yang mencakup informasi blok-blok yang membentuk suatu berkas dan lokasi penyimpanannya. Oleh karena itu, NameNode merupakan *single point of failure*. Apabila terjadi kegagalan pada NameNode, intervensi manual diperlukan untuk pemulihan.

Seluruh node slave menjalankan daemon yang disebut *DataNode*. Data aktual disimpan pada DataNode dan didistribusikan dengan cara membagi berkas menjadi blok-blok berukuran default 64 MB. NameNode menentukan pemetaan blok ke DataNode tertentu. Setiap blok direplikasi secara default sebanyak tiga salinan untuk menjamin reliabilitas dan ketersediaan. Strategi penempatan replikasi dilakukan pada beberapa rack yang berbeda guna menghindari kegagalan tingkat rack.

HDFS juga memiliki *Secondary NameNode* yang secara periodik melakukan proses *checkpoint* terhadap metadata NameNode. Namun, Secondary NameNode tidak menggantikan fungsi NameNode apabila terjadi kegagalan, sehingga sistem tetap belum sepenuhnya mendukung *high availability*. Untuk mengatasi keterbatasan kapasitas memori NameNode dalam menyimpan metadata, diperkenalkan konsep *HDFS Federation*, di mana beberapa NameNode ditambahkan dan masing-masing mengelola *namespace* secara independen.

HDFS Read/Write Operation

Proses *read* dan *write* pada HDFS melibatkan interaksi antara klien, NameNode, dan sejumlah DataNode dalam klaster. Mekanisme ini dirancang untuk mendukung distribusi data secara paralel serta menjamin reliabilitas melalui replikasi blok.



Gambar 3: *HDFS Architecture*

Operasi Write Klien HDFS memulai proses dengan mengirimkan permintaan pembuatan berkas ke Distributed File System (DFS), yang kemudian berkomunikasi dengan NameNode. NameNode melakukan verifikasi terhadap duplikasi berkas dan membuat entri metadata baru untuk blok yang akan disimpan. Selanjutnya, NameNode menentukan DataNode tujuan berdasarkan faktor replikasi (secara default tiga replika).

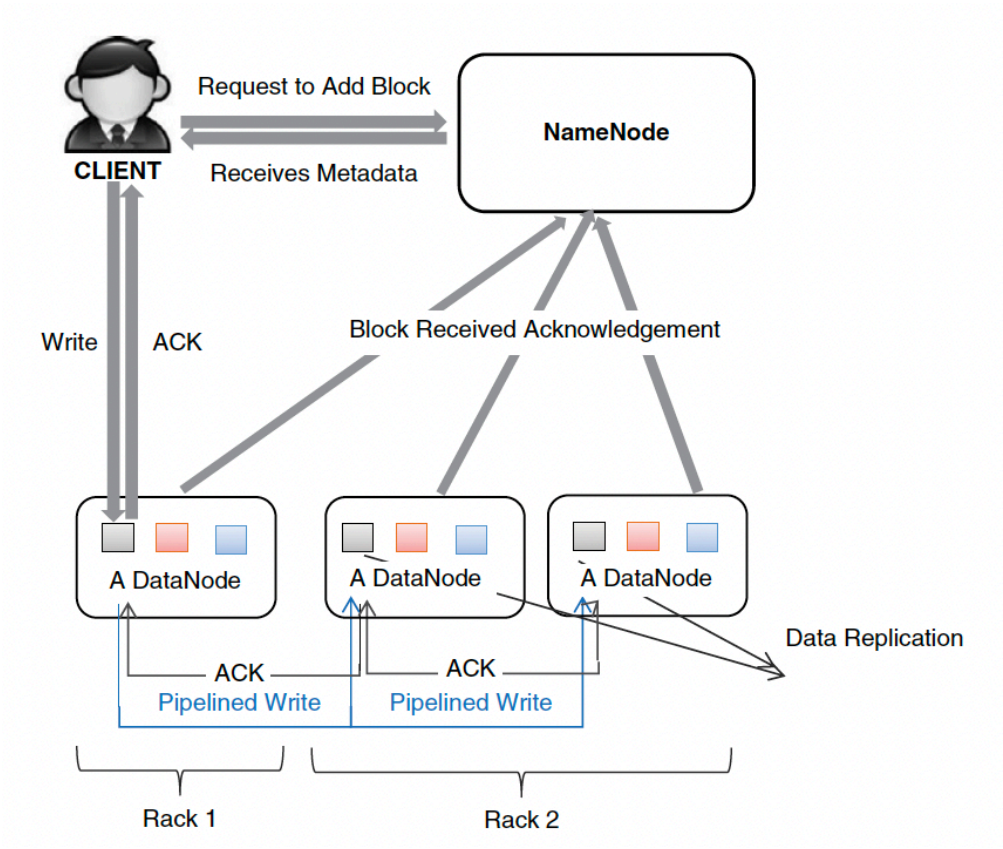
Berkas masukan dipecah menjadi blok-blok dengan ukuran default 64 MB. Blok-blok tersebut dikirim dalam bentuk paket ke DataNode secara *pipelined*. Klien terlebih dahulu mengirimkan paket ke DataNode terdekat. DataNode pertama meneruskan paket tersebut ke DataNode kedua, dan DataNode kedua meneruskannya ke DataNode ketiga. Setelah seluruh blok diterima secara lengkap, mekanisme *acknowledgement* dikirimkan secara berantai dari DataNode terakhir kembali ke klien. Jika seluruh replikasi berhasil, koneksi ditutup.

Operasi Read Pada proses pembacaan, klien mengirimkan permintaan *read* ke DFS, yang kemudian meminta metadata lokasi blok kepada NameNode. NameNode mengembalikan daftar DataNode yang menyimpan replika blok, disusun berdasarkan kedekatan jaringan (proximity).

Klien memilih DataNode terdekat dan melakukan pembacaan secara langsung. Untuk blok berikutnya, klien kembali meminta metadata ke NameNode. Proses ini berulang hingga seluruh data terbaca, kemudian koneksi ditutup. Apabila salah satu DataNode gagal, pembacaan dilakukan dari replika lain yang tersedia. Mekanisme ini memungkinkan pembacaan paralel dan meningkatkan kinerja sistem.

Rack Awareness

HDFS menerapkan mekanisme *rack awareness* untuk meningkatkan ketersediaan (*availability*) dan ketahanan terhadap ke-

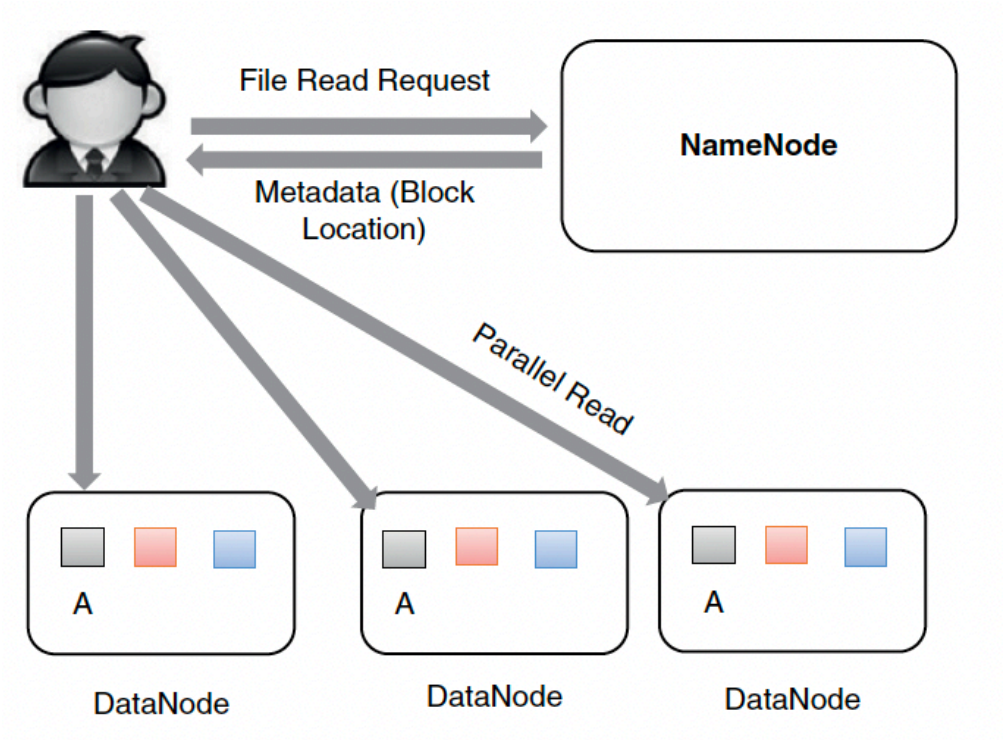


Gambar 4: *Mekanisme file write pada HDFS dengan skema pipelined replication.*

gagalan (*fault tolerance*). Dalam arsitektur klaster, DataNode tersebar pada beberapa rack yang masing-masing diidentifikasi melalui *rack ID* yang informasinya disimpan pada NameNode.

Strategi penempatan replika blok dilakukan dengan kebijakan sebagai berikut: dari tiga replika suatu blok (default replication factor = 3), replika pertama ditempatkan pada suatu rack tertentu, sedangkan replika kedua dan ketiga ditempatkan pada rack yang berbeda dari rack pertama, namun berada pada dua DataNode yang berbeda dalam rack yang sama. Dengan demikian, tidak ada lebih dari dua replika yang berada pada satu rack, dan setiap replika selalu ditempatkan pada DataNode yang berbeda.

Pendekatan ini bertujuan untuk mengatasi dua jenis kegagalan utama, yaitu kegagalan node dan kegagalan rack. Apabila rack tempat replika pertama berada mengalami kegagalan total, data masih dapat diakses dari rack lain yang menyimpan replika kedua dan ketiga. Jumlah rack yang digunakan dalam replikasi umumnya lebih sedikit daripada jumlah total replika,

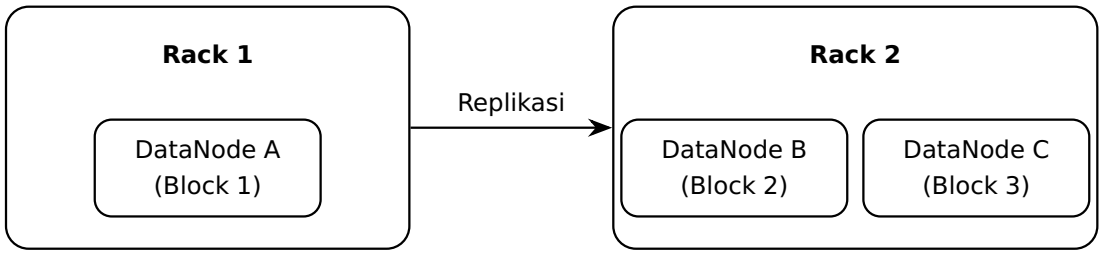


Gambar 5: *HDFS File Read Mechanism*

karena kegagalan rack relatif lebih jarang terjadi dibandingkan kegagalan DataNode.

Penempatan replika kedua dan ketiga pada rack yang sama dipertimbangkan berdasarkan efisiensi jaringan. Proses penulisan data antar DataNode dalam rack yang sama secara signifikan lebih cepat dibandingkan antar rack yang berbeda. Oleh karena itu, strategi ini mengoptimalkan kinerja penulisan sekaligus tetap mempertahankan ketahanan terhadap kegagalan.

Secara konseptual, mekanisme *rack awareness* menempatkan tiga replika blok pada tiga DataNode berbeda yang tersebar pada dua rack terpisah, sehingga mampu mengatasi kegagalan tingkat node maupun tingkat rack secara simultan.



Gambar 6: *Strategi Rack Awareness pada HDFS*

Features of HDFS

Cost-Effective HDFS merupakan platform penyimpanan bersifat *open-source*, sehingga dapat digunakan tanpa biaya lisensi oleh organisasi yang mengadopsinya. Keunggulan ekonomis HDFS tidak hanya berasal dari sifatnya yang bebas lisensi, tetapi juga dari pemanfaatan perangkat keras komoditas (*commodity hardware*) sebagai media penyimpanan. HDFS tidak memerlukan perangkat keras khusus berperforma tinggi yang umumnya berbiaya mahal. Apabila sistem penyimpanan big data bergantung pada perangkat keras kelas atas yang terspesialisasi, biaya implementasi dan operasional akan meningkat secara signifikan. Dengan memanfaatkan infrastruktur standar, HDFS menjadi solusi penyimpanan yang efisien dan hemat biaya.

Distributed Storage HDFS menerapkan mekanisme penyimpanan terdistribusi dengan cara membagi berkas masukan menjadi blok-blok berukuran default 64 MB. Setiap blok disimpan pada DataNode yang berbeda dalam klaster. Sebagai ilustrasi, berkas berukuran 200 MB akan dipecah menjadi tiga blok berukuran 64 MB dan satu blok berukuran 8 MB. Tiga blok berukuran 64 MB akan terisi penuh, sedangkan blok 8 MB tidak mengisi kapasitas blok secara keseluruhan. Sisa ruang dalam blok tersebut dapat dimanfaatkan untuk menyimpan bagian berkas lain sehingga pemanfaatan ruang penyimpanan menjadi lebih optimal. Pendekatan ini memungkinkan pengelolaan data dalam skala besar secara efisien dan paralel.

Data Replication Salah satu karakteristik utama HDFS adalah mekanisme replikasi data. Secara default, setiap blok data direplikasi sebanyak tiga salinan dan disimpan pada DataNode yang berbeda dalam klaster. Strategi ini dirancang untuk menjamin ketersediaan dan ketahanan terhadap kegagalan. Apabila terjadi kerusakan atau kegagalan pada suatu node, sistem secara otomatis mengidentifikasi node yang kehilangan salinan data dan melakukan pemulihan dengan memanfaatkan replika yang tersedia pada node lain. Jadi, integritas dan kontinuitas akses data tetap terjaga meskipun terjadi gangguan pada sebagian komponen sistem.

2.3 Hadoop Computation

MapReduce

MapReduce merupakan model pemrograman *batch-processing* pada kerangka Hadoop yang mengadopsi prinsip *divide-and-conquer*. Model ini dirancang untuk mendukung komputasi berskala besar dengan karakteristik skalabilitas tinggi, keandalan, serta toleransi terhadap kegagalan. MapReduce mampu memproses data dalam berbagai format dan bekerja dalam lingkungan komputasi paralel serta terdistribusi, meskipun secara khusus ditujukan untuk beban kerja bertipe *batch*.

Keunggulan utama MapReduce dibandingkan pendekatan tradisional terletak pada paradigma pemrosesannya. Pada sistem konvensional, data dipindahkan dari platform penyimpanan ke platform komputasi untuk dianalisis. Sebaliknya, MapReduce menjalankan komputasi langsung pada lokasi tempat data disimpan (*data locality*), sehingga mengurangi latensi dan meningkatkan efisiensi pemrosesan.

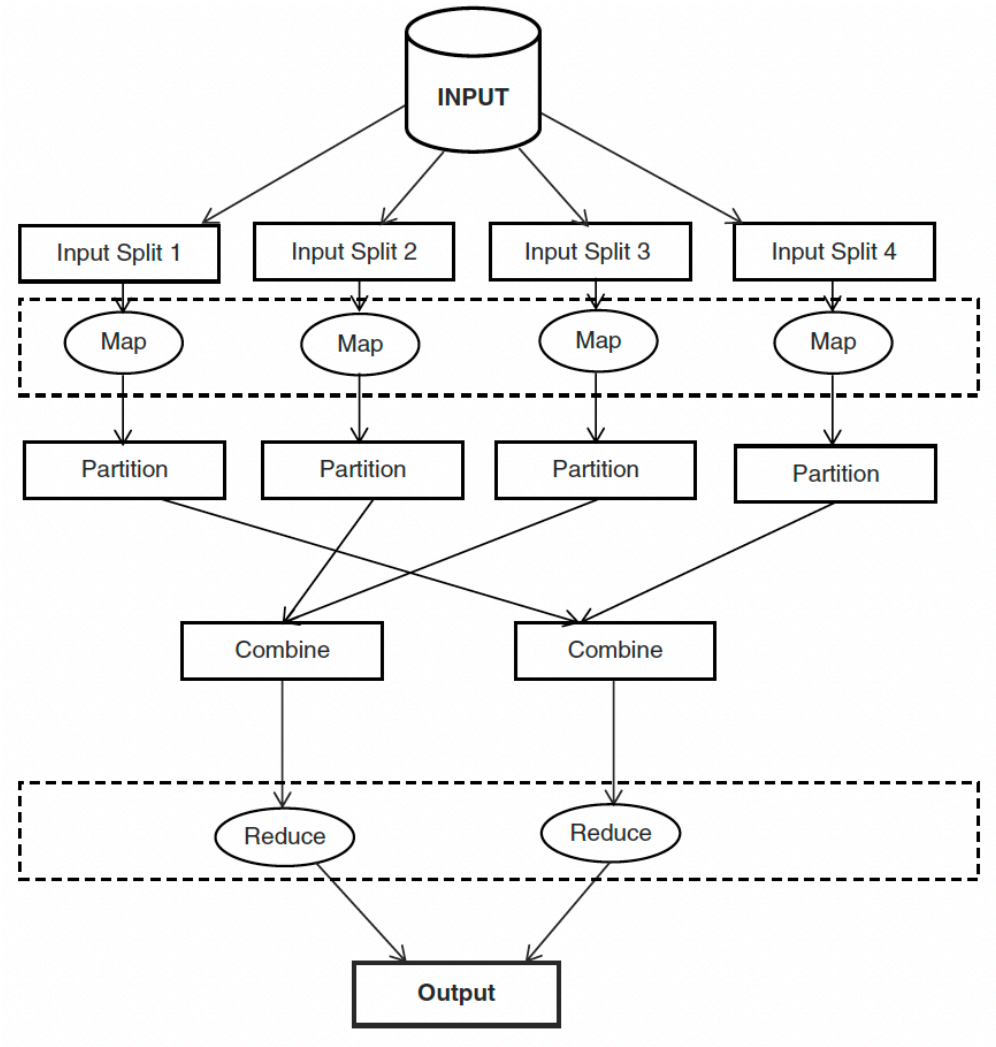
Model MapReduce membagi proses komputasi ke dalam dua fase utama, yaitu *map phase* dan *reduce phase*. Fase *map* mencakup tahapan *map*, *combine*, dan *partition*, sedangkan fase *reduce* meliputi *shuffle and sort* serta *reduce*. Komponen *combiner* dan *partitioner* bersifat opsional, bergantung pada kebutuhan logika pemrosesan data.

Dalam paradigma ini, tugas pengguna terbatas pada penyediaan program MapReduce dan data masukan. Selanjutnya, kerangka kerja Hadoop secara otomatis menangani distribusi tugas, penjadwalan, sinkronisasi, serta pengelolaan kesalahan, sehingga menyederhanakan implementasi komputasi terdistribusi.

Mapper

Tahap *map* merupakan fase awal dalam proses MapReduce. Pada tahap ini, kumpulan data berskala besar dipecah menjadi blok-blok data yang lebih kecil. Setiap blok diproses secara independen oleh *mapper* atau *map task*. Data masukan direpresentasikan dalam bentuk pasangan kunci-nilai (K_1, V_1).

Mapper menjalankan logika yang telah didefinisikan oleh pengguna dalam program MapReduce dan menghasilkan pasangan kunci-nilai antara (K_2, V_2) sebagai keluaran antara (*intermediate output*). Seluruh blok data diproses secara paralel pada node yang berbeda dalam klaster. Satu kunci dapat memiliki lebih dari satu nilai yang terkait, sehingga hasil akhir



Gambar 7: *Model MapReduce*

dari tahap ini berupa daftar pasangan (K_2, V_2) yang kemudian akan diproses pada fase berikutnya.

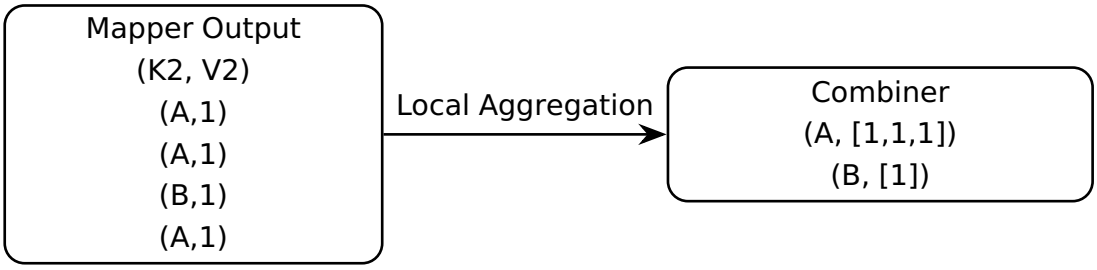
Combiner

Combiner merupakan komponen opsional dalam fase *map* yang bertujuan untuk mengoptimalkan keluaran mapper sebelum diteruskan ke tahap reducer. Tujuan utama penggunaan combiner adalah mengurangi volume data yang harus ditransfer melalui jaringan dari mapper ke reducer, sehingga menekan *overhead* komunikasi dan meningkatkan efisiensi kinerja sistem.

Secara konseptual, combiner dapat dipandang sebagai reducer lokal pada sisi map task. Combiner melakukan agregasi

awal terhadap keluaran mapper yang berupa sejumlah pasangan kunci-nilai (K_2, V_2). Pada tahap ini, kunci-kunci yang sama dikelompokkan bersama, dan nilai-nilai yang terkait dengan kunci tersebut dikombinasikan sesuai dengan logika yang ditentukan pengguna. Sebagai contoh, apabila suatu kunci muncul beberapa kali, combiner akan menggabungkan nilai-nilai tersebut menjadi satu representasi yang lebih ringkas.

Dengan demikian, data yang dikirim ke tahap *shuffle and sort* menjadi lebih kecil dibandingkan apabila seluruh keluaran mapper dikirim tanpa proses agregasi awal. Perlu dicatat bahwa penggunaan combiner bersifat opsional dan hanya dapat diterapkan apabila fungsi reduksi bersifat asosatif dan komutatif, sehingga hasil akhir tetap konsisten.



Gambar 8: *Proses Agregasi pada Combiner*

Reducer

Reducer merupakan tahap akhir dalam model MapReduce yang bertugas menjalankan fungsi logis sebagaimana didefinisikan oleh pengguna dalam program MapReduce. Setiap reducer bekerja secara independen dan terisolasi dari reducer lainnya, tanpa mekanisme komunikasi langsung antar reducer. Distribusi tugas reducer dilakukan oleh kerangka kerja Hadoop berdasarkan hasil partisi kunci pada tahap sebelumnya.

Masukan bagi reducer telah melalui proses *shuffle and sort*, sehingga data yang diterima telah dikelompokkan dan diurutkan berdasarkan kunci. Dengan demikian, reducer menerima pasangan data dalam bentuk $(K_2, [V_2])$, yaitu satu kunci dengan daftar nilai yang terkait. Reducer kemudian memproses seluruh nilai untuk setiap kunci sesuai dengan logika yang ditentukan pengguna, seperti agregasi, perhitungan statistik, atau transformasi data lainnya.

Hasil pemrosesan reducer berupa pasangan kunci-nilai baru, yang dapat mempertahankan struktur kunci awal atau menghasilkan kunci yang telah dimodifikasi sesuai kebutuhan kom-

putasi. Keluaran akhir dari reducer dituliskan kembali ke Distributed File System (DFS) sebagai hasil akhir proses MapReduce. Dengan mekanisme ini, MapReduce mampu menyelesaikan pemrosesan data berskala besar secara paralel, terdistribusi, dan terstruktur.

JobTracker and TaskTracker

Pada arsitektur Hadoop MapReduce generasi awal (MRv1), mekanisme komputasi mengadopsi pola *master/slave* yang terdiri atas satu *JobTracker* dan beberapa *TaskTracker*. JobTracker berjalan pada node master, sedangkan TaskTracker berjalan pada setiap node slave, dengan ketentuan bahwa setiap slave hanya memiliki satu TaskTracker.

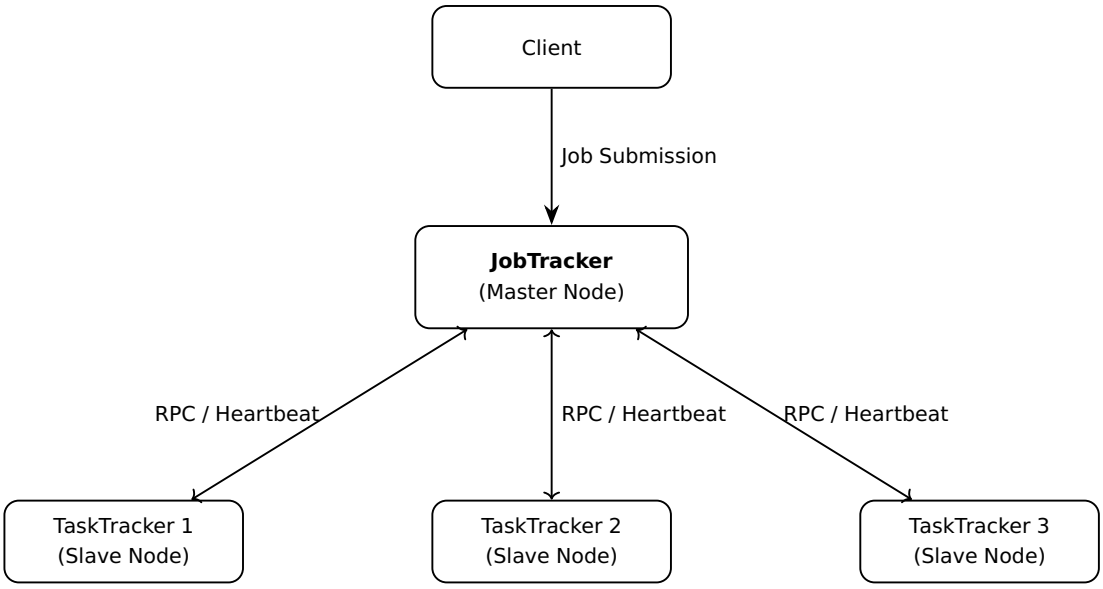
Dalam konfigurasi klaster, umumnya satu mesin menjalankan NameNode dan TaskTracker, sedangkan mesin lainnya menjalankan JobTracker dan DataNode, sehingga setiap node berperan sekaligus dalam komputasi dan penyimpanan. JobTracker bertanggung jawab atas penjadwalan tugas, pengelolaan sumber daya, pemantauan kesehatan node, serta pengawasan progres pekerjaan. Sebaliknya, TaskTracker menerima tugas dari JobTracker, mengeksekusi kode pengguna, dan mengirimkan pembaruan status secara periodik.

Komunikasi antara JobTracker dan TaskTracker berlangsung dua arah melalui mekanisme *Remote Procedure Call* (RPC). TaskTracker secara berkala mengirimkan sinyal *heartbeat* untuk menandakan bahwa node dalam keadaan aktif. Selain itu, TaskTracker juga melaporkan status tugas yang sedang berjalan atau ketersediaannya untuk menerima tugas baru. Apabila heartbeat tidak diterima dalam interval waktu tertentu, node dianggap gagal.

JobTracker merupakan *single point of failure*. Apabila JobTracker mengalami kegagalan, seluruh tugas dalam klaster berpotensi gagal. Oleh karena itu, node yang menjalankan JobTracker harus memiliki tingkat keandalan tinggi. Dalam kondisi klaster aktif dengan banyak pekerjaan berjalan secara simultan, penggunaan memori oleh JobTracker perlu dipantau secara ketat karena seluruh metadata tugas disimpan dalam memori.

Jika terjadi kegagalan eksekusi tugas, TaskTracker akan melaporkannya kepada JobTracker. JobTracker kemudian menjadwalkan ulang tugas tersebut, baik pada node yang sama maupun pada node lain dalam klaster. Apabila beberapa tugas dari pekerjaan yang sama gagal pada satu TaskTracker, maka TaskTracker tersebut tidak lagi menjalankan tugas dari pekerjaan yang bersangkutan. Jika kegagalan terjadi pada berbagai

pekerjaan yang berbeda dalam satu TaskTracker, maka node tersebut dapat dinonaktifkan sementara dari penjadwalan tugas selama periode tertentu.



Gambar 9: *Arsitektur JobTracker dan TaskTracker pada Hadoop MapReduce*

Keterbatasan MapReduce

MapReduce merupakan salah satu kerangka kerja pemrosesan paralel yang paling luas digunakan untuk menyelesaikan permasalahan berbasis data intensif, termasuk pada bidang ilmu lingkungan, keuangan, dan bioinformatika. Meskipun demikian, MapReduce memiliki sejumlah keterbatasan inheren yang berasal dari desain arsitekturnya.

Salah satu keterbatasan mendasar adalah skalabilitas satu arah (*one-way scalability*). MapReduce dirancang untuk menangani kumpulan data berukuran sangat besar secara efisien, namun kurang optimal dalam memproses sejumlah besar berkas berukuran kecil. Hal ini berkaitan dengan keterbatasan memori pada NameNode yang menyimpan metadata seluruh berkas dalam sistem. Penyimpanan metadata untuk banyak berkas kecil akan mengonsumsi memori secara signifikan, sehingga membatasi efisiensi sistem. Selain itu, NameNode berperan sebagai *single point of failure*. Apabila NameNode mengalami kegagalan, seluruh klaster menjadi tidak tersedia, sehingga sistem tidak sepenuhnya memenuhi kebutuhan *high availability* yang menjadi persyaratan penting dalam banyak aplikasi modern.

Keterbatasan lain terletak pada model eksekusi bertahap (*phase-based execution*). Fase *reduce* tidak dapat dimulai sebelum seluruh tugas pada fase *map* selesai. Demikian pula, aplikasi MapReduce berikutnya tidak dapat dieksekusi sebelum aplikasi sebelumnya menyelesaikan seluruh tahapannya. Model eksekusi yang bersifat sekuensial ini menyebabkan waktu tunggu antar tahapan dan antar pekerjaan.

Dari perspektif pemanfaatan sumber daya, pendekatan ini juga menimbulkan inefisiensi. Ketika tugas *map* sedang berjalan, node yang bertugas sebagai *reducer* berada dalam kondisi tidak aktif. Sebaliknya, ketika fase *reduce* berlangsung, node *mapper* tidak dimanfaatkan. Kondisi ini mengakibatkan utilisasi sumber daya yang tidak optimal. Selain itu, dapat terjadi situasi di mana sumber daya komputasi menganggur sambil menunggu penyelesaian fase tertentu sebelum melanjutkan ke tahap berikutnya.

Keterbatasan-keterbatasan tersebut mendorong pengembangan arsitektur yang lebih fleksibel dan mendukung ketersediaan tinggi, seperti pengenalan YARN pada generasi Hadoop berikutnya.

Perbedaan Hadoop Versi 2 dan Versi 3

Perkembangan Hadoop dari versi 2 ke versi 3 membawa sejumlah peningkatan arsitektural yang signifikan, terutama pada aspek skalabilitas, efisiensi penyimpanan, serta dukungan terhadap beban kerja modern. Berikut merupakan perbandingan utama antara Hadoop versi 2 dan Hadoop versi 3.

1. Arsitektur dan High Availability Pada Hadoop versi 2, mekanisme *High Availability* untuk NameNode telah diperkenalkan, namun masih memiliki keterbatasan dalam hal skalabilitas dan fleksibilitas manajemen metadata. Hadoop versi 3 meningkatkan dukungan *High Availability* dengan arsitektur yang lebih matang, termasuk optimalisasi HDFS Federation yang memungkinkan beberapa NameNode aktif berjalan secara independen untuk meningkatkan kapasitas dan mengurangi bottleneck metadata.

2. Erasure Coding Salah satu fitur utama yang diperkenalkan pada Hadoop versi 3 adalah *Erasure Coding*. Pada Hadoop versi 2, replikasi data dilakukan dengan faktor replikasi default tiga kali, yang menyebabkan kebutuhan penyimpanan menjadi tiga kali lipat dari ukuran data asli. Hadoop versi

3 memperkenalkan mekanisme erasure coding yang mampu mengurangi konsumsi ruang penyimpanan hingga sekitar 50% dibandingkan replikasi tradisional, tanpa mengorbankan toleransi terhadap kegagalan.

3. Skalabilitas dan Kapasitas Node Hadoop versi 2 memiliki batasan jumlah DataNode dalam satu klaster (umumnya hingga beberapa ribu node). Hadoop versi 3 meningkatkan kemampuan skalabilitas secara signifikan dengan dukungan terhadap puluhan ribu node dalam satu klaster, serta peningkatan kapasitas maksimum blok penyimpanan dan namespace.

4. Dukungan Kontainer dan YARN Pada Hadoop versi 2, YARN telah diperkenalkan sebagai pengelola sumber daya terpusat untuk memisahkan manajemen komputasi dari penyimpanan. Hadoop versi 3 meningkatkan kapabilitas YARN dengan dukungan terhadap *container orchestration*, termasuk integrasi dengan Docker dan GPU scheduling. Hal ini memungkinkan Hadoop menangani beban kerja modern seperti *machine learning* dan komputasi berbasis AI.

5. Multiple Standby NameNode Hadoop versi 3 mendukung lebih dari satu *standby NameNode*, yang meningkatkan keandalan sistem dibandingkan Hadoop versi 2 yang umumnya hanya mendukung satu pasangan aktif-standby.

6. Peningkatan Kinerja dan Manajemen Sumber Daya Versi 3 menghadirkan optimalisasi pada manajemen memori, peningkatan efisiensi *balancer*, serta perbaikan algoritma penjadwalan pada YARN. Hal ini menghasilkan utilisasi sumber daya yang lebih baik dibandingkan versi 2.

Secara umum, Hadoop versi 2 merupakan fondasi penting yang memperkenalkan YARN dan meningkatkan fleksibilitas arsitektur, sedangkan Hadoop versi 3 berfokus pada peningkatan skalabilitas, efisiensi penyimpanan melalui erasure coding, serta dukungan terhadap beban kerja komputasi modern. Dengan demikian, Hadoop versi 3 lebih sesuai untuk implementasi sistem big data skala besar yang menuntut ketersediaan tinggi dan efisiensi sumber daya.

2.4 YARN (Yet Another Resource Negotiator)

YARN (*Yet Another Resource Negotiator*) merupakan arsitektur manajemen sumber daya pada Hadoop yang memisahkan fungsi pengelolaan resource dan pemrosesan aplikasi. Tiga komponen utama YARN adalah:

- ResourceManager
- ApplicationMaster
- NodeManager

ResourceManager

ResourceManager adalah komponen tunggal (one-per-cluster) yang bertanggung jawab atas alokasi sumber daya ke berbagai aplikasi dalam cluster. ResourceManager terdiri dari dua komponen utama, yaitu *ApplicationsManager* dan *Scheduler*.

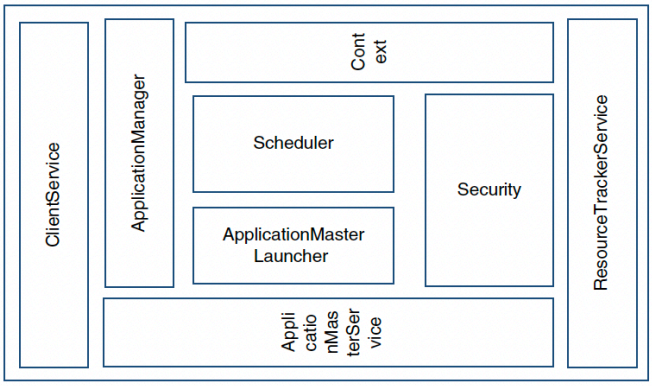
- **ApplicationsManager** Mengelola ApplicationMaster di seluruh cluster. Bertanggung jawab untuk:
  - Menerima atau menolak aplikasi yang diajukan.
  - Menyediakan resource awal untuk ApplicationMaster.
  - Memantau status aplikasi yang sedang berjalan.
  - Melakukan restart aplikasi jika terjadi kegagalan.
- **Scheduler** Mengalokasikan resource berdasarkan kebijakan:
  - FIFO
  - Fair Scheduling
  - Capacity Scheduling

  Scheduler tidak memantau status aplikasi; tugasnya hanya melakukan alokasi resource sesuai kebutuhan aplikasi.

Selain dua komponen utama tersebut, ResourceManager juga memiliki beberapa service pendukung:

- **ClientService** – Antarmuka untuk client dalam melakukan submit, terminate, dan monitoring aplikasi.

- **ApplicationMasterService** – Menangani komunikasi RPC dari ApplicationMaster.
- **ResourceTrackerService** – Menangani negosiasi resource antara ApplicationMaster dan ResourceManager.
- **ApplicationMasterLauncher** – Meluncurkan container untuk ApplicationMaster saat job disubmit.
- **Security Component** – Menghasilkan *ContainerToken* dan *ApplicationToken* untuk keamanan akses.



Gambar 10: *ResourceManager*

Diagram Arsitektur ResourceManager

NodeManager

Gambar 12 memperlihatkan berbagai komponen yang terdapat pada NodeManager. Komponen *NodeStatusUpdater* bertugas membangun komunikasi antara ResourceManager dan NodeManager, serta memperbarui informasi kepada ResourceManager mengenai status container yang sedang berjalan pada node tersebut.

ContainerManager bertanggung jawab mengelola seluruh container yang aktif di dalam node. Sementara itu, *ContainerExecutor* berinteraksi langsung dengan sistem operasi untuk menjalankan maupun membersihkan proses container sesuai dengan instruksi yang diberikan.

Komponen *NodeHealthCheckerService* memantau kondisi kesehatan node dan secara periodik mengirimkan sinyal *heartbeat* kepada ResourceManager sebagai indikator bahwa node dalam keadaan aktif dan berfungsi dengan baik. Pada aspek

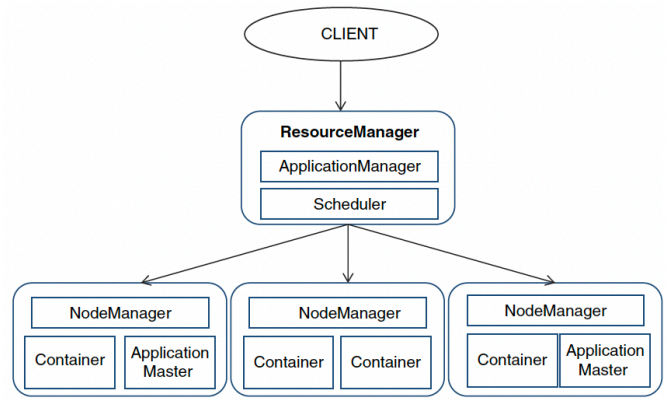
keamanan, komponen *Security* memastikan bahwa setiap permintaan yang masuk telah terotorisasi oleh ResourceManager sebelum diproses lebih lanjut.

Arsitektur YARN

Pada arsitektur Hadoop 1.0, framework MapReduce hanya mendukung pemrosesan secara *batch*. Untuk melakukan pemrosesan data secara *real-time* maupun *near real-time*, data harus diekstraksi dari Hadoop dan dipindahkan ke sistem basis data lain. Untuk mengatasi keterbatasan tersebut, Yahoo mengembangkan YARN (*Yet Another Resource Negotiator*).

Gambar 11 menunjukkan arsitektur YARN yang tidak lagi menggunakan komponen JobTracker dan TaskTracker. Sebagai gantinya, YARN terdiri atas tiga komponen utama, yaitu *ResourceManager*, *ApplicationMaster*, dan *NodeManager*.

Tanggung jawab yang sebelumnya dipegang oleh JobTracker—meliputi alokasi sumber daya, penjadwalan pekerjaan, dan pemantauan—dibagi antara ResourceManager dan ApplicationMaster. ResourceManager bertugas mengalokasikan seluruh sumber daya cluster kepada aplikasi yang berjalan. ApplicationMaster bersama NodeManager melaksanakan serta memantau eksekusi aplikasi.



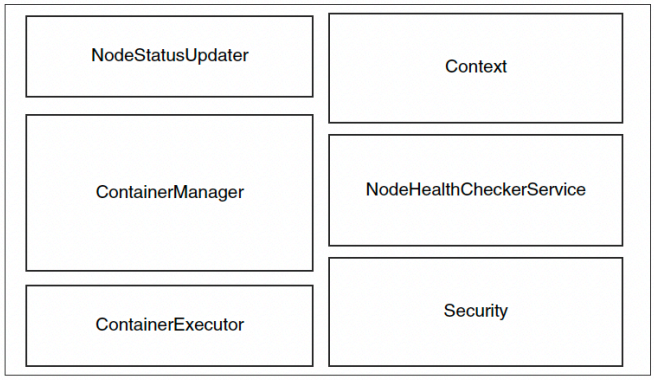
Gambar 11: *Arsitektur YARN*

ResourceManager dilengkapi dengan *pluggable scheduler* yang memungkinkan alokasi sumber daya kepada aplikasi berdasarkan kebijakan tertentu. Namun, berbeda dengan JobTracker pada Hadoop MapReduce, ResourceManager tidak melakukan pemantauan langsung terhadap status tugas dan tidak memiliki mekanisme toleransi kesalahan untuk memulai ulang tugas yang gagal akibat gangguan perangkat keras atau kegagalan aplikasi.

ApplicationMaster melakukan negosiasi sumber daya dengan ResourceManager serta melacak status penggunaan sumber daya tersebut. NodeManager memantau pemakaian sumber daya pada setiap node dan melaporkannya kepada ResourceManager.

Dalam YARN, aplikasi mengajukan permintaan sumber daya melalui ApplicationMaster. Scheduler kemudian merespons permintaan tersebut dengan memberikan *container*. Container merupakan unit alokasi sumber daya sebagai respons terhadap *ResourceRequest*, yang merepresentasikan hak aplikasi untuk menggunakan sejumlah sumber daya tertentu dalam cluster.

Karena fungsi alokasi sumber daya, penjadwalan pekerjaan, dan pemantauan telah ditangani oleh ResourceManager, ApplicationMaster, dan NodeManager, MapReduce tetap dapat digunakan untuk pemrosesan tanpa perubahan signifikan. Dalam konteks ini, MapReduce berperan sebagai kerangka pemrosesan yang berjalan di atas YARN. Selain MapReduce, berbagai kerangka kerja lain juga dapat memanfaatkan YARN untuk menjalankan proses komputasi. Dengan demikian, YARN bersifat lebih generik dibandingkan arsitektur Hadoop MapReduce sebelumnya karena mendukung pemrosesan tugas non-MapReduce yang tidak didukung pada Hadoop MapReduce generasi awal.



Gambar 12: *NodeManager*

YARN Scheduler

Arsitektur YARN dilengkapi dengan komponen *scheduler* yang bertugas mengalokasikan sumber daya cluster sesuai dengan kebutuhan aplikasi berdasarkan kebijakan penjadwalan tertentu. Beberapa kebijakan penjadwalan yang tersedia dalam YARN

meliputi:

- FIFO Scheduler
- Capacity Scheduler
- Fair Scheduler

FIFO Scheduler

FIFO (*First In, First Out*) merupakan kebijakan penjadwalan yang paling sederhana dan mudah diimplementasikan. Dalam pendekatan ini, pekerjaan dieksekusi berdasarkan urutan waktu pengajuan, di mana aplikasi yang disubmit lebih dahulu akan diproses terlebih dahulu tanpa mempertimbangkan prioritasnya.

Setiap pekerjaan ditempatkan dalam sebuah antrian. Pekerjaan yang berada pada posisi pertama dalam antrian akan dijalankan terlebih dahulu. Setelah pekerjaan tersebut selesai, pekerjaan berikutnya dalam antrian akan diproses, dan seterusnya hingga seluruh antrian terselesaikan.

Kebijakan FIFO bekerja secara efektif untuk pekerjaan berdurasi singkat karena waktu tunggu relatif kecil. Namun, apabila terdapat pekerjaan dengan waktu eksekusi yang panjang, sebagian besar sumber daya cluster dapat terserap oleh pekerjaan tersebut. Akibatnya, pekerjaan lain—terutama yang berukuran kecil— harus menunggu lebih lama hingga sumber daya tersedia kembali.

Capacity Scheduler

Capacity Scheduler memungkinkan beberapa aplikasi berbagi sumber daya cluster secara aman dan terkontrol sehingga setiap aplikasi memperoleh alokasi yang proporsional. Pendekatan ini diimplementasikan dengan mengonfigurasi satu atau lebih antrian (*queues*), di mana setiap antrian diberikan porsi kapasitas tertentu dari total sumber daya cluster.

Struktur antrian dapat dibagi secara hierarkis, sehingga beberapa aplikasi dapat berbagi kapasitas yang dialokasikan untuk suatu antrian tertentu. Di dalam masing-masing antrian, mekanisme penjadwalan biasanya mengikuti kebijakan FIFO.

Setiap antrian dilengkapi dengan *Access Control List* (ACL) yang menentukan pengguna mana yang berhak mengajukan pekerjaan ke antrian tertentu. Apabila terdapat lebih dari satu

pekerjaan yang berjalan dalam suatu antrian dan masih tersedia sumber daya yang belum terpakai, scheduler dapat mengalokasikan sumber daya tersebut ke pekerjaan lain dalam antrian yang sama.

Fair Scheduler

Fair Scheduler merupakan kebijakan penjadwalan yang dirancang untuk mendistribusikan sumber daya cluster secara adil kepada seluruh aplikasi yang sedang berjalan dalam suatu periode waktu tertentu. Prinsip utamanya adalah memberikan porsi sumber daya yang relatif seimbang kepada setiap aplikasi aktif.

Apabila suatu aplikasi sedang menggunakan sebagian besar sumber daya cluster dan kemudian aplikasi lain diajukan, Fair Scheduler akan mengalokasikan sumber daya yang tersedia agar kedua aplikasi memperoleh bagian yang seimbang. Mekanisme *preemption* juga diterapkan, yaitu penghentian sementara eksekusi suatu aplikasi sehingga container yang dialokasikan dapat dikembalikan dan diberikan kepada aplikasi lain yang membutuhkan.

Dalam pendekatan ini, setiap antrian dapat diberi bobot (*weight*) tertentu. Antrian dengan bobot ringan memperoleh alokasi minimum sumber daya, sedangkan antrian dengan bobot besar memperoleh alokasi sumber daya yang lebih tinggi. Saat mengajukan aplikasi, pengguna dapat memilih antrian sesuai dengan kebutuhan sumber daya. Aplikasi dengan kebutuhan sumber daya besar dapat diajukan ke antrian berbobot tinggi, sedangkan aplikasi ringan dapat menggunakan antrian berbobot rendah.

Sebagai ilustrasi, apabila sebuah pekerjaan besar dijalankan dan menjadi satu-satunya pekerjaan dalam cluster, seluruh sumber daya akan dialokasikan kepadanya. Jika kemudian pekerjaan kecil diajukan, sebagian sumber daya dari pekerjaan besar akan dialihkan agar kedua pekerjaan memperoleh porsi yang relatif setara. Setelah pekerjaan kecil selesai, seluruh sumber daya kembali dialokasikan kepada pekerjaan besar. Dengan mekanisme ini, pemanfaatan sumber daya cluster menjadi lebih efisien dan penyelesaian pekerjaan dapat dilakukan secara tepat waktu.

Failures in YARN

Keberhasilan eksekusi suatu aplikasi pada Hadoop 2.0 sangat bergantung pada koordinasi berbagai komponen YARN, yaitu

ResourceManager, *NodeManager*, *ApplicationMaster*, serta *container*. Kegagalan pada salah satu komponen tersebut dapat menyebabkan kegagalan aplikasi secara keseluruhan. Mengingat Hadoop merupakan kerangka kerja terdistribusi, penanganan kegagalan dalam sistem semacam ini relatif lebih kompleks dan membutuhkan mekanisme toleransi kesalahan yang terstruktur.

Secara umum, jenis kegagalan dalam YARN dapat diklasifikasikan menjadi:

- Kegagalan ResourceManager
- Kegagalan ApplicationMaster
- Kegagalan NodeManager
- Kegagalan Container

ResourceManager Failure

Pada versi awal YARN, ResourceManager merupakan *single point of failure*. Apabila terjadi kegagalan, diperlukan intervensi manual untuk melakukan penelusuran kesalahan dan memulai ulang layanan tersebut. Selama ResourceManager tidak aktif, seluruh cluster tidak dapat digunakan. Setelah ResourceManager aktif kembali, seluruh pekerjaan yang sedang berjalan pada ApplicationMaster harus dijalankan ulang.

Untuk mengatasi permasalahan ini, arsitektur YARN dikembangkan dengan dua pendekatan peningkatan keandalan. Pertama, penerapan mekanisme *active-passive ResourceManager*. Dalam skema ini, apabila ResourceManager aktif mengalami kegagalan, ResourceManager pasif akan secara otomatis mengambil alih peran pengelolaan cluster.

Pendekatan kedua adalah penggunaan *ZooKeeper* untuk menyimpan status (ResourceManager state). Ketika ResourceManager aktif gagal, kondisi tersebut dideteksi dan diinformasikan kepada ResourceManager pasif melalui ZooKeeper, yang kemudian mengubah statusnya menjadi aktif dan melanjutkan pengelolaan cluster.

ApplicationMaster Failure

Kegagalan ApplicationMaster dideteksi oleh ResourceManager. Dalam kondisi ini, ResourceManager akan menyediakan container baru dan menjalankan instans ApplicationMaster yang baru untuk melakukan percobaan ulang eksekusi aplikasi.

ApplicationMaster yang baru bertanggung jawab memulihkan status dari ApplicationMaster sebelumnya. Proses pemulihan hanya dapat dilakukan apabila status aplikasi disimpan pada lokasi eksternal. Jika informasi status tidak tersedia, maka aplikasi akan dijalankan kembali dari awal.

NodeManager Failure

NodeManager berjalan pada setiap node slave dan bertanggung jawab atas eksekusi sebagian tugas aplikasi. NodeManager secara periodik mengirimkan sinyal *heartbeat* kepada ResourceManager sebagai indikator bahwa node dalam keadaan aktif.

Apabila ResourceManager tidak menerima sinyal *heartbeat* dalam interval waktu tertentu, NodeManager dianggap mengalami kegagalan dan dikeluarkan dari cluster. Kegagalan ini kemudian dilaporkan kepada ApplicationMaster, dan container yang berjalan pada NodeManager tersebut dihentikan. ApplicationMaster akan menjadwalkan ulang bagian tugas yang sebelumnya dijalankan pada NodeManager yang gagal tersebut.

Container Failure

Container bertanggung jawab menjalankan tugas *map* dan *reduce*. Kegagalan container terdeteksi oleh ApplicationMaster ketika tidak ada respons yang diterima dari container dalam periode waktu tertentu. Dalam situasi ini, ApplicationMaster akan mencoba mengeksekusi ulang tugas tersebut.

Apabila tugas yang sama gagal berulang kali hingga melebihi batas yang telah ditentukan, maka tugas tersebut dianggap gagal secara permanen. Jumlah percobaan ulang dapat dikonfigurasi oleh pengguna, baik berdasarkan jumlah maksimum percobaan ulang maupun berdasarkan persentase kegagalan tugas selama eksekusi pekerjaan berlangsung.

3 Konfigurasi Lab

3.1 Konfigurasi Lingkungan Laboratorium Big Data

Pada praktikum ini, mahasiswa akan membangun sebuah lingkungan *single-node Hadoop cluster* menggunakan Docker dalam

mode *pseudo-distributed*. Konfigurasi ini memungkinkan simulasi arsitektur komputasi terdistribusi pada satu mesin lokal tanpa memerlukan infrastruktur multi-node secara fisik. Pendekatan ini efektif untuk pembelajaran konsep dasar Hadoop seperti HDFS, YARN, serta manajemen proses terdistribusi.

3.2 Tujuan Konfigurasi

Konfigurasi ini bertujuan untuk:

1. Menyediakan lingkungan eksperimen Hadoop yang terisolasi dan reprodusibel menggunakan teknologi *containerization*.
2. Memahami proses deployment sistem big data berbasis Docker.
3. Melakukan eksplorasi layanan inti Hadoop seperti *NameNode*, *DataNode*, dan *ResourceManager*.

3.3 Prasyarat Sistem

Sebelum memulai proses instalasi, pastikan sistem telah memenuhi kebutuhan berikut:

1. **Docker Desktop** Docker digunakan sebagai platform virtualisasi berbasis container untuk menjalankan Hadoop cluster. Unduh dan instal melalui: `https://www.docker.com/products/docker-desktop`
2. **Akun Docker** Login ke Docker Desktop menggunakan akun Docker Hub. Jika belum memiliki akun, lakukan pendaftaran terlebih dahulu.
3. **Windows Subsystem for Linux (WSL) + Ubuntu** Sistem dijalankan melalui lingkungan Linux (Ubuntu) pada Windows. Jalankan PowerShell sebagai Administrator dan eksekusi:

   ```
   wsl --install -d Ubuntu-24.04
   ```

4. **Akses Terminal** Buka terminal Ubuntu (WSL), lalu arahkan ke direktori drive Windows:

   ```
   cd /mnt/c
   ```

Alternatif menggunakan Command Prompt:

```
cd C:\
```

3.4 Prosedur Instalasi dan Deployment

Setelah seluruh prasyarat terpenuhi, lakukan tahapan berikut secara sistematis:

1. **Clone Repository** Unduh source code konfigurasi Hadoop dari GitHub:

   ```
   git clone https://github.com/sains-data/bigdata-hadoop.git
   ```

2. **Normalisasi Format File** Buka folder `bigdata-hadoop` menggunakan VS Code. Pastikan seluruh file menggunakan format akhir baris **LF (Linux Format)**. Ubah dari `CRLF` ke `LF` melalui indikator kanan bawah VS Code.

3. **Masuk ke Direktori Project**

   ```
   cd /mnt/c/bigdata-hadoop
   ```

   atau pada terminal Windows:

   ```
   cd C:\bigdata-hadoop
   ```

4. **Menambahkan File Distribusi Hadoop**

   File `hadoop-3.4.1.tar.gz` tidak diunduh otomatis oleh Dockerfile. Unduh secara manual dari: https://downloads.apache.org/hadoop/common/hadoop-3.4.1/hadoop-3.4.1.tar.gz

   Pindahkan file tersebut ke dalam folder `bigdata-hadoop`.

   Opsional: Untuk otomatisasi, tambahkan baris berikut ke dalam Dockerfile:

   RUN curl -O https://downloads.apache.org/hadoop/common/hadoop-3.4.1/hadoop-3.4.1.tar.gz

5. **Membangun Docker Image** Proses ini akan melakukan build image Hadoop berdasarkan Dockerfile:

   ```
   bash build.sh
   ```

6. **Menjalankan Container Hadoop**

```
bash start.sh
```

7. **Verifikasi Status Container**

   ```
   docker ps -a
   ```

   Pastikan container dalam status `Up`.

8. **Login ke dalam Container**

   ```
   bash login.sh
   ```

9. **Verifikasi Layanan Hadoop**

   ```
   jps
   ```

   Pastikan proses berikut berjalan:

   - NameNode
   - DataNode
   - ResourceManager
   - NodeManager

10. **Mengakses Web Interface**

    Setelah layanan aktif, akses melalui browser:

    - Hadoop NameNode UI: `http://localhost:9870`
    - Hadoop Resource Manager UI: `http://localhost:8088`

11. **Keluar dari Container**

    ```
    exit
    ```

12. **Menghentikan Container**

    ```
    bash stop.sh
    ```

Konfigurasi ini menggunakan pendekatan *pseudo-distributed mode*, di mana seluruh komponen Hadoop berjalan dalam satu container, namun tetap mensimulasikan mekanisme sistem terdistribusi seperti komunikasi antar daemon dan manajemen resource melalui YARN. Pendekatan ini cukup untuk eksperimen dasar seperti:

- Operasi HDFS (put, get, mkdir, ls)

- Eksekusi MapReduce sederhana
- Monitoring cluster melalui Web UI

Untuk kebutuhan produksi atau simulasi skala besar, disarankan menggunakan arsitektur multi-node berbasis Docker Compose atau cluster berbasis VM terpisah.

4 Latihan

Latihan ini dirancang untuk mengasah pemahaman mahasiswa terhadap konsep HDFS, MapReduce, dan YARN yang telah dibahas pada sesi konsep dasar. Setiap kegiatan mencakup eksplorasi langsung di dalam lingkungan Hadoop yang telah dikonfigurasi pada sesi sebelumnya. Pastikan container Hadoop telah berjalan sebelum memulai.

4.1 Eksplorasi HDFS (15 Menit)

Pada kegiatan ini, mahasiswa akan melakukan operasi dasar pada *Hadoop Distributed File System* (HDFS) untuk memahami bagaimana data disimpan, didistribusikan, dan dikelola dalam sistem terdistribusi.

Langkah-Langkah

1. **Masuk ke dalam container Hadoop.**

   ```
   bash login.sh
   ```

2. **Buat direktori kerja di HDFS.** Buat sebuah direktori baru bernama `/user/latihan` di dalam HDFS untuk menyimpan data praktikum:

   ```
   hdfs dfs -mkdir -p /user/latihan
   ```

3. **Buat file teks lokal sebagai data uji.** Di dalam container, buat sebuah file teks sederhana yang akan diunggah ke HDFS:

   ```
   echo "Hadoop adalah framework big data terdistribusi.
   HDFS menyimpan data dalam blok berukuran 64MB.
   MapReduce memproses data secara paralel di klaster.
   YARN mengelola sumber daya pada setiap node.
   NameNode menyimpan metadata sistem berkas HDFS.
   DataNode menyimpan blok data aktual di setiap node."
   > latihan.txt
   ```

4. **Unggah file ke HDFS.** Pindahkan file yang telah dibuat ke direktori HDFS yang telah disiapkan:

   ```
   hdfs dfs -put latihan.txt /user/latihan/
   ```

5. **Verifikasi file berhasil diunggah.** Tampilkan isi direktori `/user/latihan` untuk memastikan file berhasil tersimpan di HDFS:

   ```
   hdfs dfs -ls /user/latihan/
   ```

6. **Tampilkan isi file dari HDFS.** Baca isi file secara langsung dari HDFS tanpa mengunduhnya ke penyimpanan lokal:

   ```
   hdfs dfs -cat /user/latihan/latihan.txt
   ```

7. **Periksa informasi blok file.** Amati bagaimana HDFS membagi dan menyimpan file dalam bentuk blok beserta informasi replikasinya:

   ```
   hdfs fsck /user/latihan/latihan.txt -files -blocks -locations
   ```

8. **Unduh file dari HDFS ke lokal.** Ambil kembali file dari HDFS dan simpan dengan nama berbeda di direktori lokal container:

   ```
   hdfs dfs -get /user/latihan/latihan.txt hasil_unduh.txt
   cat hasil_unduh.txt
   ```

Pertanyaan Refleksi

1. Berapa ukuran blok default yang digunakan HDFS berdasarkan keluaran perintah `fsck`? Mengapa HDFS menggunakan pembagian blok dalam penyimpanan data?

2. Berapa jumlah salinan (*replication factor*) yang diterapkan pada file yang Anda unggah? Apa yang terjadi pada data jika salah satu *DataNode* mengalami kegagalan?

3. Jelaskan perbedaan antara perintah `hdfs dfs -put` dan `hdfs dfs -get`. Komponen Hadoop mana yang berperan dalam menentukan lokasi penyimpanan blok saat operasi *put* berlangsung?

4.2 Pemantauan Klaster melalui Web UI (10 Menit)

Hadoop menyediakan antarmuka web untuk memantau kondisi klaster secara *real-time*. Pada kegiatan ini, mahasiswa akan mengeksplorasi informasi klaster melalui *NameNode UI* dan *ResourceManager UI*.

Langkah-Langkah

1. **Akses NameNode Web UI.** Buka browser dan navigasikan ke alamat berikut untuk memantau kondisi HDFS:

   `http://localhost:9870`

   Pada halaman utama, perhatikan informasi berikut:

   - **Overview**: Status *NameNode*, kapasitas total HDFS, dan kapasitas yang telah digunakan.
   - **Datanodes**: Daftar *DataNode* yang aktif beserta kapasitas penyimpanannya masing-masing.
   - **Utilities** → **Browse the file system**: Navigasikan ke direktori `/user/latihan` dan verifikasi file yang telah diunggah pada Kegiatan 1.

2. **Akses ResourceManager Web UI.** Buka tab baru dan navigasikan ke:

   `http://localhost:8088`

   Amati informasi berikut:

   - **Cluster Metrics**: Jumlah node aktif, sumber daya yang tersedia, dan kapasitas memori klaster.
   - **Applications**: Daftar aplikasi YARN yang pernah dijalankan (*finished*, *running*, atau *failed*).
   - **Nodes**: Status setiap *NodeManager* yang terdaftar dalam klaster.

3. **Catat informasi klaster.** Lengkapi tabel berikut berdasarkan data yang ditampilkan pada kedua Web UI:

Informasi	Nilai
Versi Hadoop	
Kapasitas HDFS (Total)	
Kapasitas HDFS (Digunakan)	
Jumlah DataNode Aktif	
Total Memori Klaster (YARN)	
Jumlah NodeManager Aktif	

Pertanyaan Refleksi

1. Pada *NameNode UI*, apa yang dimaksud dengan kolom *Configured Capacity* dan *DFS Used*? Mengapa kedua nilai ini penting untuk dipantau?

2. Berdasarkan tampilan *ResourceManager UI*, apa fungsi *NodeManager* dalam arsitektur YARN?

4.3 Eksekusi Program MapReduce — Word Count (25 Menit)

Word Count merupakan program paling fundamental dalam paradigma MapReduce, yang menghitung frekuensi kemunculan setiap kata dalam sekumpulan dokumen teks. Pada kegiatan ini, mahasiswa akan menjalankan program *Word Count* bawaan Hadoop terhadap data yang telah diunggah ke HDFS pada Kegiatan 1.

Latar Belakang Konseptual

Program *Word Count* dengan MapReduce bekerja dalam dua fase utama:

- **Fase Map**: Setiap baris teks dipecah menjadi kata-kata individual. Mapper menghasilkan pasangan kunci-nilai $(kata, 1)$ untuk setiap kata yang ditemukan. Sebagai contoh, kalimat *"Hadoop adalah framework"* akan menghasilkan keluaran `(Hadoop, 1)`, `(adalah, 1)`, dan `(framework, 1)`.

- **Fase Reduce**: Seluruh pasangan dengan kunci yang sama dikumpulkan, kemudian nilai-nilainya dijumlahkan. Sebagai contoh, jika kata *"Hadoop"* muncul tiga kali di seluruh dokumen, maka *reducer* akan menghasilkan `(Hadoop, 3)`.

Langkah-Langkah

1. **Buat dataset yang lebih besar untuk MapReduce.** Di dalam container, buat file teks berisi beberapa baris untuk menghasilkan frekuensi kata yang lebih variatif:

```
cat <<EOF > dataset_wordcount.txt
hadoop hdfs mapreduce yarn hadoop
namenode datanode hadoop hdfs blok
mapreduce mapper reducer hadoop yarn
hdfs replikasi blok namenode hadoop
yarn resourcemanager nodemanager hadoop
mapreduce hadoop hdfs paralel terdistribusi
EOF
```

2. **Unggah dataset ke HDFS.**

```
hdfs dfs -mkdir -p /user/latihan/input
hdfs dfs -put dataset_wordcount.txt /user/latihan/input/
```

3. **Verifikasi file berhasil diunggah.**

```
hdfs dfs -ls /user/latihan/input/
```

4. **Jalankan program Word Count bawaan Hadoop.** Eksekusi MapReduce menggunakan library *hadoop-mapreduce-examples* yang telah tersedia di dalam instalasi Hadoop:

```
hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/\
hadoop-mapreduce-examples-*.jar \
wordcount /user/latihan/input /user/latihan/output
```

5. **Pantau progres job di YARN.** Saat job berjalan, buka browser dan amati halaman `http://localhost:8088`. Perhatikan status aplikasi yang sedang berjalan, termasuk progres fase *map* dan *reduce*.

6. **Verifikasi hasil keluaran.** Setelah job selesai, tampilkan direktori hasil dan baca isi file keluarannya:

```
hdfs dfs -ls /user/latihan/output/
hdfs dfs -cat /user/latihan/output/part-r-00000
```

7. **Unduh hasil ke lokal.**

```
hdfs dfs -get /user/latihan/output/part-r-00000 \
hasil_wordcount.txt
cat hasil_wordcount.txt
```

Pertanyaan Refleksi

1. Berdasarkan hasil *Word Count*, kata apa yang paling sering muncul? Apakah hasilnya sesuai dengan isi dataset yang Anda buat?

2. Pada output di terminal saat job berjalan, terdapat informasi persentase progres *map* dan *reduce* secara terpisah. Mengapa kedua fase ini dimonitor secara terpisah? Apa yang terjadi antara selesainya fase *map* dan dimulainya fase *reduce*?

3. Mengapa direktori *output* HDFS harus dalam kondisi **belum ada** sebelum menjalankan job MapReduce? Apa yang terjadi jika direktori tersebut sudah ada?

4.4 Manajemen Data dan Pembersihan HDFS (10 Menit)

Pada kegiatan ini, mahasiswa akan mempraktikkan operasi manajemen data di HDFS, termasuk penggantian nama, penyalinan, pemindahan, serta penghapusan berkas. Kemampuan ini penting untuk pengelolaan data dalam skenario produksi.

Langkah-Langkah

1. **Salin file di dalam HDFS.** Buat salinan file `latihan.txt` dengan nama baru tanpa mengunduhnya ke lokal terlebih dahulu:

```
hdfs dfs -cp /user/latihan/latihan.txt \
/user/latihan/latihan_backup.txt
hdfs dfs -ls /user/latihan/
```

2. **Pindahkan file ke direktori lain.** Buat direktori baru dan pindahkan file backup ke dalamnya:

   ```
   hdfs dfs -mkdir /user/latihan/arsip
   hdfs dfs -mv /user/latihan/latihan_backup.txt \
   /user/latihan/arsip/
   hdfs dfs -ls /user/latihan/arsip/
   ```

3. **Cek penggunaan ruang penyimpanan.** Periksa berapa ruang HDFS yang digunakan oleh direktori latihan secara keseluruhan:

   ```
   hdfs dfs -du -h /user/latihan/
   ```

4. **Periksa ringkasan kapasitas HDFS.**

   ```
   hdfs dfsadmin -report
   ```

   Perhatikan informasi *Configured Capacity*, *DFS Used*, dan *DFS Remaining* pada output perintah ini.

5. **Hapus direktori output MapReduce.** Setelah memverifikasi hasilnya, hapus direktori output untuk membebaskan ruang:

   ```
   hdfs dfs -rm -r /user/latihan/output
   hdfs dfs -ls /user/latihan/
   ```

Pertanyaan Refleksi

1. Berdasarkan keluaran perintah `hdfs dfsadmin -report`, berapa *live DataNode* yang terdaftar dalam klaster? Bagaimana informasi ini mencerminkan arsitektur *single-node* yang digunakan pada praktikum ini?

2. Apakah HDFS mendukung operasi *update* atau modifikasi isi file yang sudah diunggah? Jelaskan alasannya berdasarkan prinsip *write-once, read-many* yang telah dibahas pada sesi konsep dasar.